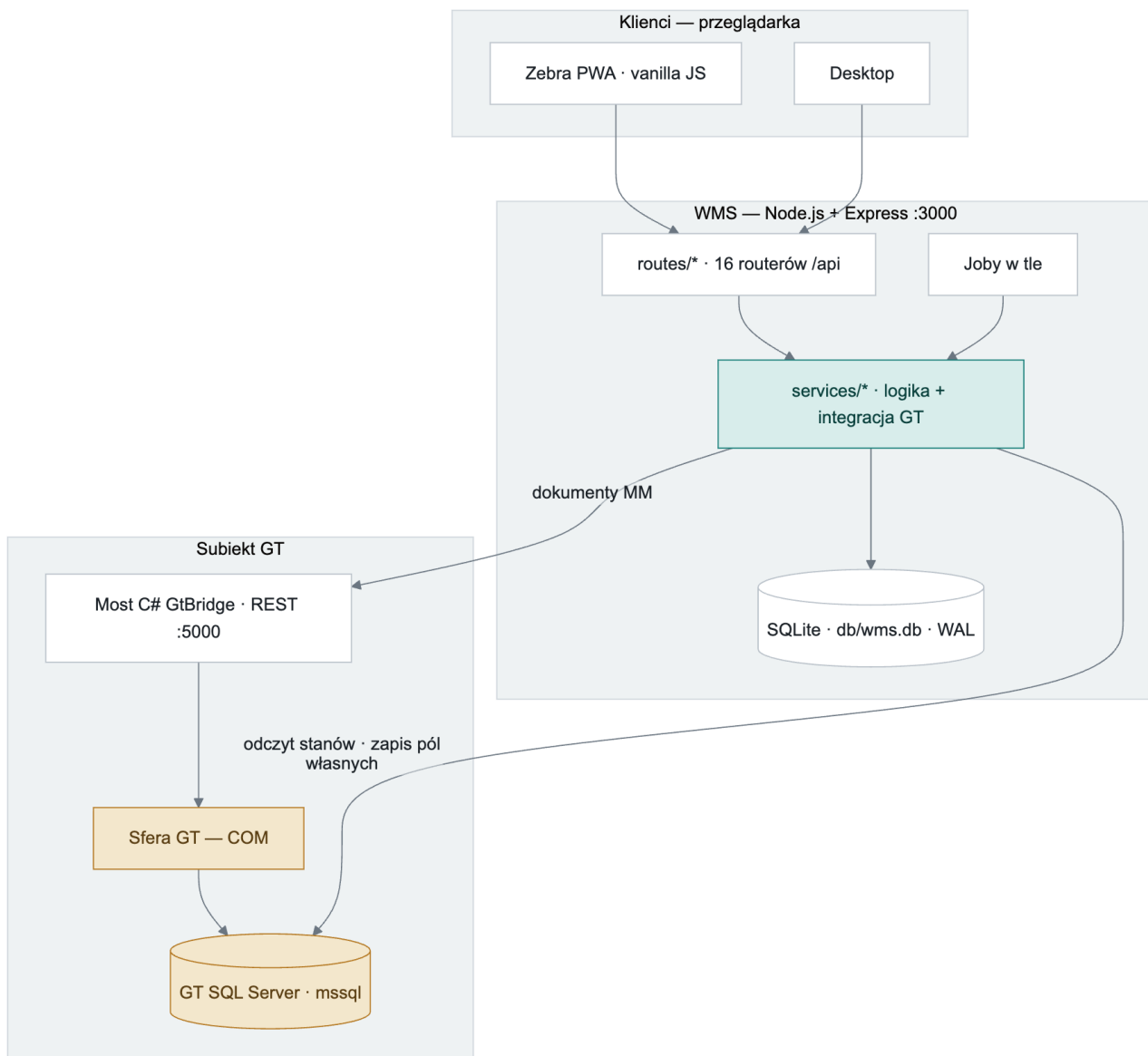


Architektura WMS

Lekki WMS jako uzupełnienie Subiekt GT — obsługuje lokalizacje magazynowe, przesunięcia MM i parametry produktu. Zbiór i wysyłkę robi Sellasist. Ten dokument rysuje **jak to jest poskładane**; reguły biznesowe (co wolno, a czego nie) są w zasady.md, a pełny kontekst dla agenta w ../CLAUDE.md.

Diagramy są w mermaid — renderują się na GitHubie i w edytorach z podglądem mermaid (VS Code: rozszerzenie „Markdown Preview Mermaid Support”).

1. Kontekst systemu



Rys. 1 · Kontekst systemu

- **Frontend** to statyczne pliki serwowane przez Express (`app.js`, `express.static` z `Cache-Control: no-cache`). Zebra i Desktop to ta sama apka w dwóch layoutach. Brak frameworka i brak buildu — czysty HTML + vanilla JS. Skan na Zebrze wstrzykuje DataWedge do aktywnego `<input>` (patrz zasady.md).

- **Backend** — Express montuje 16 routerów pod `/api/*`, logikę trzyma w `services/*`, stan lokalny w SQLite (wbudowany `node:sqlite`, tryb WAL, synchroniczny).
- **GT** — dwa niezależne kanały (sekcja 3).

2. Warstwy i moduły

Warstwa	Gdzie	Rola
Klienci	<code>public/zebra</code> , <code>public/desktop</code>	UI; walidacja tylko dla UX, nieautorytatywna
Trasy	<code>routes/*</code>	HTTP <code>/api/*</code> , egzekwują inwarianty (jedyne źródło prawdy reguł)
Logika domenowa	<code>services/*</code> (bez <code>gt-*</code>)	rozbicie stanu, kanały rezerwacji, model lokalizacji, uzupełnienia, strefy
Integracja GT	<code>services/gt-*</code>	most C# + bezpośredni SQL do GT
Dane lokalne	<code>db/database.js</code> , <code>db/001_init.sql</code>	SQLite: lokalizacje, stany, ruchy, audyt
Joby	<code>services/*-job.js</code> i pokrewne	cykliczna synchronizacja (sekcja 5)

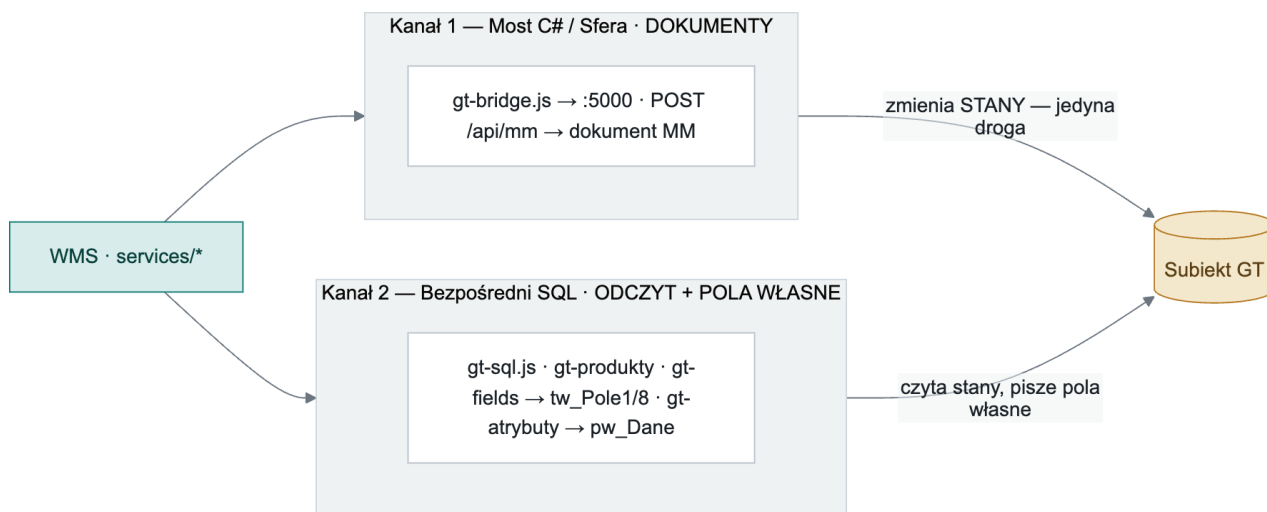
Trasy pogrupowane wg domeny (montaż w `app.js`):

- **Ruch i lokalizacje:** `ruchy` (LOK/MM, kolejka `pending`), `lokalizacje` (dom K4, skan, model kodu)
- **Rozkładanie dostaw/zwrotów:** `zwroty`, `dostawy`, `uzupelnienia`, `do-sprawdzenia`
- **Obchody magazynu:** `sciezki` (Ostatnie sztuki / K4 rezerwacja / Czyść zera / Brak parametrów)
- **Produkt i stany:** `produkty` (parametry → pola własne), `magazyny`, `zestawienia`
- **Panel/nadzór:** `pulpit`, `rozjazdy`, `audyt`, `status`
- **Dostęp:** `uzytkownicy` (logowanie), `blokady` (blokada edycji)

Kluczowe serwisy domenowe (pełna lista w `services/`): `lokalizacje-model` (`rozbierzKod` → `hala/regal/typ`), `rozbicie-stanu` i `do-rozlozenia` (kubelki rozkładania), `kanaly` (rozbicie rezerwacji na kanały), `adnotacja-stref` (dopisek `+StD/+StZ`), `wozek-model`, `wyszukiwanie`, `uzupelnienia`.

3. Integracja z GT — dwa kanały

To jest sedno architektury. **Stany zmienia się TYLKO dokumentem przez Sferę; pola własne pisze się bezpośrednim SQL-em. Nigdy odwrotnie.**



Rys. 2 · Dwa kanały do GT

Kanał 1 — Most C# (`services/gt-bridge.js`, domyślnie `http://localhost:5000`). Wystawia dokumenty MM przez Sferę GT (COM). To **jedyna** droga, którą WMS zmienia stany. Klient mostu nigdy nie rzuca na błędzie sieci/HTTP — zwraca `{ok, status, dane, blad}`, żeby trasa mogła zostawić ruch w kolejce jako `pending` (sekcja 4). Adres nadpisujący przez `GT_BRIDGE_URL` (w konfiguracji testowej celowo wskazuje martwy port, żeby nie wystawić MM).

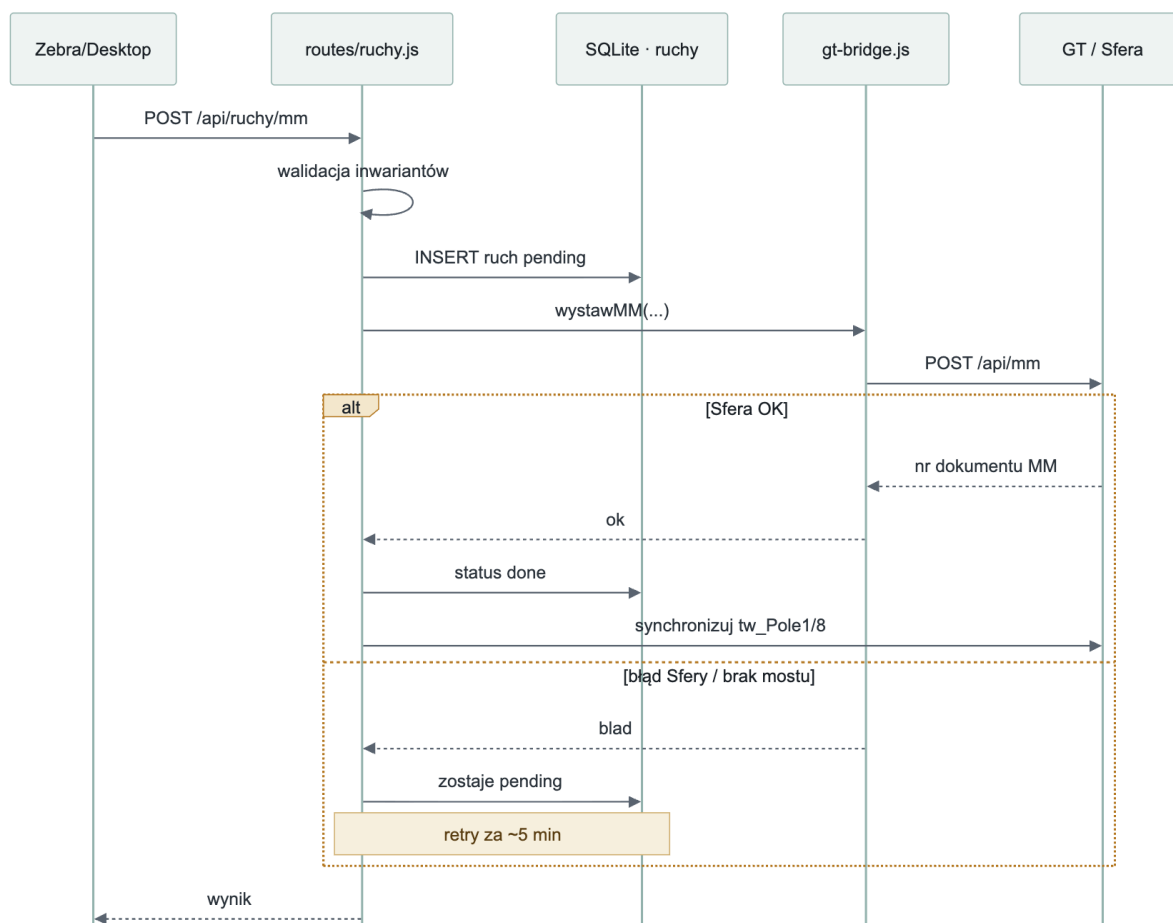
Kanał 2 — Bezpośredni SQL (`services/gt-sql.js`, `mssql`). Czyta stany i kartotekę (`gt-produkty.js`) oraz **pisze pola własne**:

- `gt-fields.js` → `UPDATE tw__Towar.tw_Pole1 / tw_Pole8` (lokalizacje K4 / K4G),
- `gt-atrybuty.js` → `UPSERT pw_Dane` (wymiar, wagi) z alokacją `pwd_Id` przez procedurę `spIdentyfikator` (**nigdy** `MAX(pwd_Id)+1` — to psuło licznik GT, patrz `zasady.md`).

▲ Nagłówek w `gt-sql.js` mówi „tylko do odczytu” — to nieaktualne. Bezpośredni SQL **pisze** pola własne (lokalizacje, parametry). Nie pisze **nigdy** stanów ilościowych. Most-owy stub `ZapiszLokalizacjeAsync` jest niepodłączony — pola własne idą wyłącznie kanałem 2.

4. Przepływ ruchu (kolejka `pending`)

Każdy ruch najpierw ląduje w SQLite ze statusem `pending`, dopiero potem woła most. Przy błędzie Sfery ruch **zostaje** `pending` i czeka na retry — nie ginie.



Rys. 3 · Przepływ ruchu (kolejka pending)

Walidacja inwariantów (ilość ≤ stan lokalizacji, cel w innym magazynie, ilość ≤ stan GT – rezerwacja itd.) dzieje się **w trasie, przed** zapisem — pełna lista w zasady.md. Front waliduje te same reguły tylko dla szybkiego feedbacku.

5. Joby w tle

Startowane w `app.js` (linie 89–96). Interwały nadpiswalne z `.env`.

Job	Plik	Interwał	Co robi	Pisze do GT?
Retry ruchów	<code>ruchy-retry.js</code>	5 min	ponawia MM w statusie <code>pending</code>	tak (most)
Rozjazdy	<code>rozjazdy.js</code>	10 min · <code>ROZJAZDY_INTERWAL_MIN</code>	detekcja GT↔WMS; auto-korekta K4 (1 lok.)	kopia WMS
Strefy w GT	<code>strefy-w-gt-job.js</code>	10 min · <code>WMS_STREFY_INTERWAL_MIN</code>	dopisek stref do <code>tw_Pole1</code> (tylko przy zmianie)	tak (<code>tw_Pole1</code>)
Rozmontowania	<code>rozmontowania.js</code>	10 min · <code>WMS_ROZMONTOWANIA_INTERWAL_MIN</code>	auto-dopis rozmontowań zestawów	—
Waga gabarytowa	<code>waga-gabarytowa-job.js</code>	6 h · <code>WAGA_GAB_INTERWAL_MIN</code>	przelicza wagę gab. po ręcznej zmianie wymiarów w Subieckie	tak (<code>pw_Dane</code>)
Reconciliacja MM	<code>reconciliacja-mm.js</code>	1 h	uzgadnia MM wystawione mostem z GT	odczyt
Snapshot pulpitu	<code>pulpit-snapshot.js</code>	1 h	migawka statusów dla dashboardu	— (SQLite)

Job	Plik	Interwał	Co robi	Pisze do GT?
Backup	<code>backup.js</code>	co godz. 7–20 + 2:00	kopia <code>wms.db</code> , rotacja dziadek-ojciec-syn	— (lokalny, <code>WMS_BACKUP_DISABLED</code>)

Joby podpisują wpisy audytu jako `system:<job>` i są domyślnie ukryte w Logu zmian (rozpoznanie po prefiksie użytkownika — patrz `zasady.md`).

6. Model danych (SQLite)

Schemat w `db/001_init.sql`, migracje inline w `db/database.js` (dodają kolumny przy starcie, gdy brakuje).

Tabela	Rola
<code>lokalizacje</code>	katalog miejsc K4/K4G; cechy (<code>hala/regal/typ</code>) liczone z kodu przez <code>lokalizacje-model.js</code>
<code>stany_lokalizacji</code>	ile czego leży na danej lokalizacji (kopia — GT jest master stanów) + <code>artykul_ean</code>
<code>ruchy</code>	kolejka ruchów LOK/MM ze statusem; kolumny źródła: <code>mag_zrodlo_zewnetrzny</code> , <code>mag_zrodlo_pula</code> , <code>zrodlo_dok</code>
<code>rozjazdy</code>	wykryte rozbieżności GT vs WMS do decyzji magazyniera
<code>audyt</code>	log zdarzeń (sprawdzenia ścieżek, zmiany, akcje jobów)

Pola po stronie GT, którymi zarządza WMS (kopie do wyświetlenia): `tw_Pole1` (miejsce K4), `tw_Pole8` (lokalizacje K4G), `pw_Dane` (wymiar/wagi). Szczegóły i „kto pisze” — `zasady.md`.

7. Uwierzytelnianie i dostęp

- `POST/PUT/DELETE` wymagają sesji (`auth.wymagajSesjiNaZapisie`) — middleware **wstrzykuje operatora z tokenu do `req.body`**, więc handlersy nie muszą znać „kto”. `GET` jest otwarty.
- Rola `uczeń` ma zawężony dostęp (`auth.blokujUcznia(...)`) — m.in. tylko zwroty i wybrane ścieżki; ścieżka „Czyść zera” jest przed nim ukryta (jako jedyna kasuje dane).
- Blokada edycji (`services/blokady.js`, `blokady.middlewareRuch`) — twarda blokada na desktapie.

8. Uruchomienie

`node app.js` → serwer na `:3000`, / przekierowuje do Zebry (`/zebra/ruch.html`). Na macOS: `start-wms.command` / `stop-wms.command`. Konfiguracja GT (host/port/hasło SQL, adres mostu) w `.env` (poza repo; wzór w `.env.example`).

Uwaga przy lokalnym uruchamianiu do testów: joby z tabeli w sekcji 5 **piszą do GT** (kanał 2 pisze pola własne, most wystawia MM). Konfiguracja testowa neutralizuje most (`GT_BRIDGE_URL` na martwy port) i backup (`WMS_BACKUP_DISABLED=1`), ale bezpośredni SQL nadal celuje w bazę z `.env`. Szczegóły w `MEMORY.md` → „MINA: testowy pisze do produkcji”.